Project 1 - Multilayer perceptron

Olha Morozova

1 Model Selection and Data Preparation

The aim of this project is to find an optimal configuration of hyper-parameters for a neural network classifier using a two-stage tuning process. At the beginning of the training, we load the datasets. Each data sample is represented as a feature vector, and its corresponding class label. Before training, we **normalize** inputs by subtracting the mean and dividing by the standard deviation for each dimension. This ensures that all features have a similar scale and prevents certain features from dominating others during the training.

The dataset contains inputs from three different classes. Figure 1 shows a 2D plot of the samples with each class separated. This can be a relatively simple classification task in low-dimensional space. We assume that a neural network with a small number of parameters should be sufficient to achieve high accuracy ($\geq 95\%$). Therefore, we keep training limits strict to achieve faster convergence and select efficient configurations.

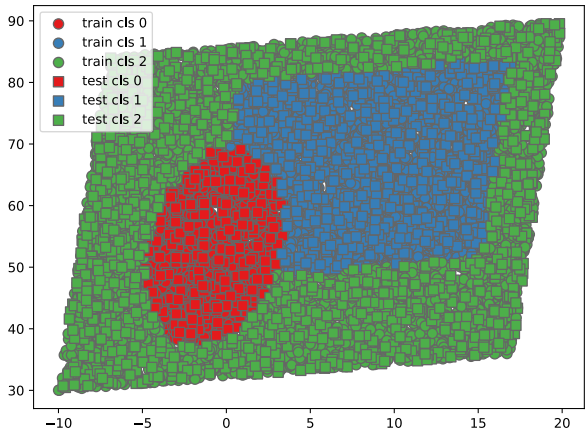

Figure 1: Training and test data visualized by class.

Since our training dataset contains 8000 samples, we use a **mini-batch learning** with

a batch size of **128**. Each epoch consists of multiple updates, one for each mini-batch. Originally, the model was trained with an online learning algorithm (batch size = 1), but mini-batch reduced training time while keeping convergence accuracy.

To evaluate model generalization during training, we split the original training dataset into two parts: **estimation set** (80%) and **validation set** (20%). For reproducibility, we use a fixed random seed `np.random.seed(42)`.

1.1 Activation Function Comparison

To select the most suitable activation function for the hidden layer, we compared four standard options:

- Sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x))$
- Tanh: $\tanh(x), \quad \tanh'(x) = 1 - \tanh^2(x)$
- ReLU: $f(x) = \max(0, x), \quad f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$
- Leaky ReLU: $f(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}, \quad f'(x) = \begin{cases} 1 & x > 0 \\ 0.01 & x \leq 0 \end{cases}$

For a fixed set of parameters ($\alpha = 0.05, \text{hidden} = 10, \text{batch\_size} = 128$), we plotted the classification error over training epochs for each activation function.

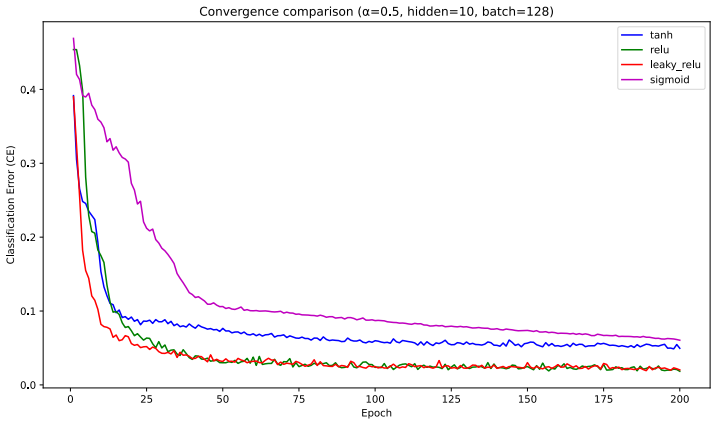

Figure 2: Comparison of activation functions: classification error over training epochs.

From the graph, we see that `tanh` and `relu` show faster convergence and lower final error than `sigmoid` and `leaky_relu`. The best option of activation will be still tested during next steps.

1.2 Early Stopping

To avoid overfitting and reduce unnecessary computation, we apply the early stopping method. During training, we evaluate the model on the validation set. If the validation performance does not improve for a fixed number of epochs, training is stopped early.

We set the following parameters: maximum number of **epochs**: 200, **patience**: 10 epochs. This means that if the validation classification error does not improve for 10 epochs, training is stopped early, and the model is restored to the best-performing state on the validation set.

2 Stage 1: Hyper-Parameter Selection

During first testing phase, we perform a grid search over the following hyper-parameters:

- Learning rate: $\alpha \in \{0.05, 0.1, 0.2, 0.5\}$
- Number of hidden units: $\{5, 7, 8\}$
- Activation function: tanh, relu, leaky_relu, sigmoid

The goal is to verify that the model performs well even with a small number of hidden units, which allows faster training. Although the learning rate values are relatively high, we aim to quickly converge to a solution and test whether such values are suitable. For each configuration, we record the classification error on the estimation and validation sets and track the epoch where early stopping occurred.

Table 1 shows the results for each configuration sorted by validation error.

Table 1: Results from the first stage of hyper-parameter testing.

alpha	hidden	est_CE	val_CE	activation	best_epoch
0.5	8	0.0378125	0.0325	relu	89
0.5	7	0.048125	0.036875	relu	81

alpha	hidden	est_CE	val_CE	activation	best_epoch
0.5	8	0.04546875	0.038125	tanh	107
0.2	7	0.04234375	0.03875	leaky_relu	94
0.2	8	0.04125	0.040625	relu	119
0.5	7	0.05734375	0.045625	leaky_relu	50
0.1	8	0.0471875	0.0475	tanh	160
0.2	5	0.054375	0.05125	relu	177
0.5	5	0.05828125	0.0525	relu	59
0.2	8	0.0509375	0.056875	tanh	120
0.5	7	0.051875	0.05875	sigmoid	117
0.2	8	0.0628125	0.06	leaky_relu	125
0.5	8	0.0575	0.06125	sigmoid	146
0.1	8	0.06828125	0.073125	leaky_relu	121
0.1	8	0.07046875	0.075	relu	94
0.1	7	0.0725	0.075625	leaky_relu	126
0.5	7	0.089375	0.078125	tanh	46
0.5	5	0.07921875	0.08625	leaky_relu	25
0.1	5	0.09640625	0.086875	relu	169
0.05	8	0.07796875	0.088125	leaky_relu	99
0.2	7	0.09109375	0.09125	tanh	140
0.5	5	0.1003125	0.096875	sigmoid	136
0.1	5	0.0984375	0.098125	leaky_relu	112
0.2	8	0.1078125	0.105625	sigmoid	195
0.2	5	0.10390625	0.11	leaky_relu	96
0.5	5	0.11	0.11125	tanh	26
0.2	7	0.12296875	0.1175	sigmoid	197
0.1	5	0.113125	0.118125	tanh	150
0.2	5	0.11875	0.1325	tanh	147
0.5	8	0.15171875	0.15375	leaky_relu	65
0.05	5	0.20953125	0.205	tanh	179
0.1	7	0.223125	0.2275	relu	33
0.1	7	0.2225	0.234375	sigmoid	200
0.1	5	0.2403125	0.255625	sigmoid	175
0.05	8	0.285	0.2775	relu	9
0.05	8	0.285	0.296875	tanh	12
0.05	7	0.29390625	0.305	tanh	29
0.05	5	0.32	0.326875	leaky_relu	9

alpha	hidden	est_CE	val_CE	activation	best_epoch
0.05	7	0.39328125	0.381875	relu	1
0.05	5	0.38546875	0.394375	relu	19
0.1	7	0.40640625	0.41875	tanh	4
0.05	5	0.4615625	0.439375	sigmoid	4
0.2	5	0.46640625	0.445625	sigmoid	2
0.05	7	0.45390625	0.445625	sigmoid	1
0.2	7	0.4596875	0.445625	relu	8
0.05	8	0.4834375	0.445625	sigmoid	3
0.05	7	0.456875	0.445625	leaky_relu	2
0.1	8	0.45390625	0.445625	sigmoid	1

The best configuration selected after Stage 1 was: $\alpha = 0.5$, hidden $= 8$, activation $=$ `relu`, best_epoch $= 89$ with val_CE $= 0.0325$, est_CE $= 0.0378125$. Both values are below 5%. The selection was based on the validation CE, choosing the configuration with the lowest error.

3 Stage 2: Features and Regularization

In the second tuning phase, we expand our model configuration by testing additional features, that influence how the learning progresses over time: momentum, L2 regularization, learning rate schedule.

Momentum This method helps accelerate gradients in the correct direction and decrease oscillations. It uses a fraction γ of the previous gradient in the current update:

$$\Delta W_t = \gamma \cdot \Delta W_{t-1} + \nabla W_t,$$
$$W \leftarrow W + \alpha_t \cdot \Delta W_t$$

The momentum parameter γ controls how much of the previous gradient is used. **Tested values** were $\gamma \in \{0.0, 0.5, 0.9\}$.

L2 Regularization This adds a penalty term to the loss function based on the size of the model weights: $\text{Loss}_{\text{L2}} = \text{Loss}_{\text{original}} + \lambda \cdot \sum w^2$.

During backpropagation, it modifies the weight update rule: $\frac{\partial \text{Loss}}{\partial W} \leftarrow \frac{\partial \text{Loss}}{\partial W} - \lambda \cdot W$, where λ is the regularization strength. This helps prevent overfitting by putting off large weight values. **Tested values** were $\lambda \in \{0.0, 0.01, 0.1\}$.

Learning Rate Scheduling Instead of keeping learning rate α_t fixed, a schedule can be applied to reduce the learning rate over time. We tested four types of schedules:

- Constant: $\alpha_t = \alpha_0$
- Linear decay: $\alpha_t = \alpha_0 \cdot \left(1 - \frac{t}{T}\right)$
- Geometric decay: $\alpha_t = \alpha_0 \cdot \gamma^t$
- Step decay: $\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/k \rfloor}$

where $\gamma = 0.9$ and step size $k = 10$.

3.1 Training

We use the best-performing parameters' set from Stage 1. Each combination of momentum, regularization strength, and learning rate schedule is evaluated on the validation set. As in the previous phase, we apply early stopping with patience = 10 and record the stopping epoch.

The model selects the configuration with the lowest val_CE as the final model setup. Table 2 summarizes the outcomes for each tested combination.

Table 2: Stage 2 results: testing momentum, L2 regularization and learning rate schedules.

alpha	hidden	activation	momentum	l2	lr_sch	est_CE	val_CE	best_epoch
0.5	8	relu	0.9	0.0	linear	0.03796875	0.033125	16
0.5	8	relu	0.9	0.0	constant	0.0471875	0.03375	16
0.5	8	relu	0.9	0.0	step	0.05265625	0.03625	17
0.5	8	relu	0.0	0.0	linear	0.038125	0.043125	33
0.5	8	relu	0.5	0.0	linear	0.04765625	0.04625	34
0.5	8	relu	0.5	0.0	step	0.0503125	0.046875	17
0.5	8	relu	0.0	0.0	step	0.04796875	0.049375	58

alpha	hidden	activation	momentum	l2	lr_sch	est_CE	val_CE	best_epoch
0.5	8	relu	0.0	0.0	constant	0.05515625	0.050625	36
0.5	8	relu	0.5	0.0	constant	0.0890625	0.095625	15
0.5	8	relu	0.5	0.0	geometric	0.09578125	0.10125	25
0.5	8	relu	0.0	0.0	geometric	0.1075	0.114375	38
0.5	8	relu	0.9	0.0	geometric	0.149375	0.15125	23
0.5	8	relu	0.5	0.01	constant	0.30796875	0.275	3
0.5	8	relu	0.9	0.01	constant	0.29859375	0.283125	4
0.5	8	relu	0.5	0.01	step	0.3003125	0.284375	3
0.5	8	relu	0.0	0.01	step	0.3	0.29	3
0.5	8	relu	0.0	0.01	linear	0.2846875	0.293125	10
0.5	8	relu	0.9	0.01	geometric	0.2903125	0.29875	14
0.5	8	relu	0.9	0.01	linear	0.3209375	0.300625	2
0.5	8	relu	0.9	0.01	step	0.30171875	0.309375	13
0.5	8	relu	0.0	0.01	constant	0.30390625	0.310625	64
0.5	8	relu	0.0	0.01	geometric	0.30546875	0.315625	20
0.5	8	relu	0.5	0.01	linear	0.31953125	0.324375	15
0.5	8	relu	0.5	0.01	geometric	0.3215625	0.33	17
0.5	8	relu	0.5	0.1	step	0.45390625	0.445625	1
0.5	8	relu	0.5	0.1	linear	0.45390625	0.445625	1
0.5	8	relu	0.5	0.1	constant	0.45390625	0.445625	1
0.5	8	relu	0.9	0.1	geometric	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	step	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	geometric	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	linear	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	constant	0.45390625	0.445625	1
0.5	8	relu	0.9	0.1	constant	0.45390625	0.445625	1
0.5	8	relu	0.9	0.1	linear	0.45390625	0.445625	1
0.5	8	relu	0.5	0.1	geometric	0.45390625	0.445625	1
0.5	8	relu	0.9	0.1	step	0.45390625	0.445625	1

The best-performing configuration was: $\alpha = \mathbf{0.5}$, hidden = **8**, activation = **relu**, momentum = **0.9**, l2_lambda = 0, lr_schedule = **linear**, best_epoch = **16** with est_CE = **0.03797**, val_CE = **0.033125**.

4 Final Model Training and Testing

In the final stage, the model was trained on the full training set using the best configuration. Since no validation data is available, the original early stopping mechanism cannot be used. Instead, we trained the model for a number of epochs proportional to the best epoch found earlier. We set the number of epochs at best_epoch $\times$ 1.25, since the estimation set used before was 80% of the training data. This aims to prevent overfitting while using the full available data.

The final model was trained using: activation function: **ReLU**; hidden units: **8**; learning rate α: **0.5**; momentum: **0.9**; L_2 regularization: 0.0; learning rate schedule: **linear**; best epoch (from validation): 16; final training epochs: **20**.

Results After training, the model was evaluated on the test set. The classification error (CE) and regression error (RE) were: **Test CE**: 2.95%; **Test RE**: 0.06299.

Confusion Matrix The confusion matrix below shows the classification accuracy per class. Each row represents the true class and each column the predicted class. The values are given in percentages:

True / Pred	A	B	C
A	88.73	2.82	8.45
B	1.27	97.14	1.59
C	0.64	0.18	99.17

Table 3: Confusion Matrix.

The matrix shows that the model performs well across all classes, especially on class C. Figure 3 shows the final classification on the test set. The predicted labels closely match the true labels.

Figure 4 displays the classification and regression errors during training. We can observe an improvement in the model over the epochs with decreasing CE.

Figure 5 shows the training and test accuracy per epoch.

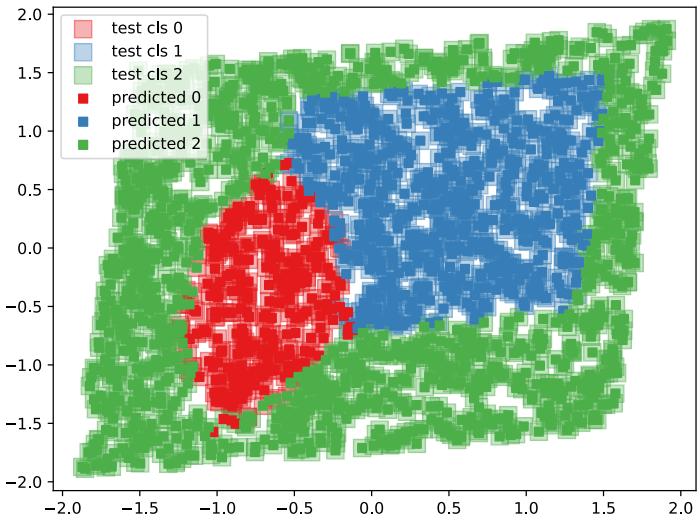

Figure 3: Classification of test data.

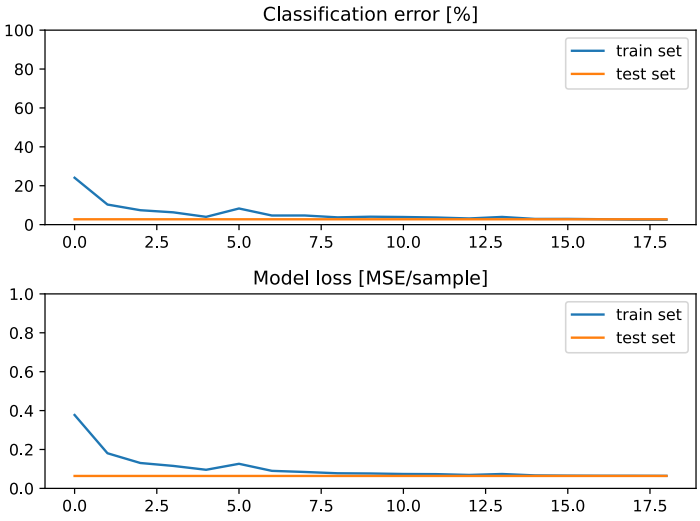

Figure 4: Training CE and RE over epochs.

Conclusion The final model demonstrates good performance on the test set, achieving a classification error below 5%. The validation-based selection of parameters ensured that the model remains efficient and does not overfit.

5 Optional: Training with Adam Optimizer

As an alternative we test the performance of the MLP classifier when trained using the Adam optimizer. Adam (Adaptive Moment Estimation) is known for requiring less tuning

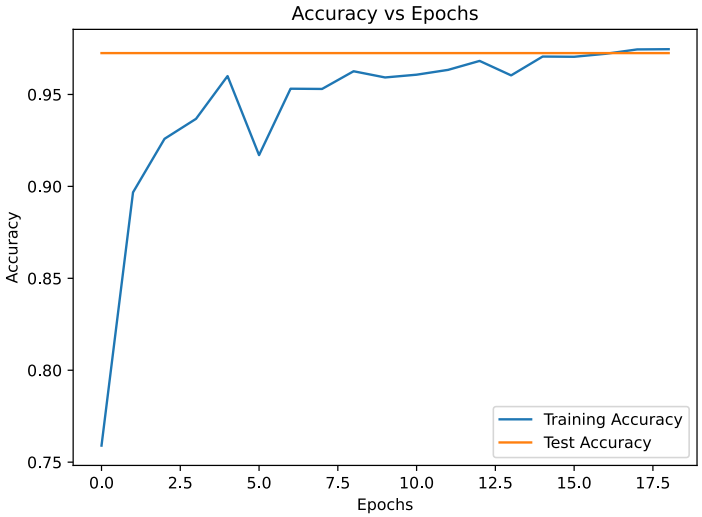

Figure 5: Training and test accuracy over epochs.

and typically works well.

In this phase, we keep the model setup and hyper-parameters from the best configuration found in earlier stages. We perform a grid search over the following additional parameters: L2 penalty values: 0.0, 0.01, 0.1; learning rate schedules: constant, linear, geometric, step. Momentum is not used here, since Adam already includes it by tracking exponentially decaying averages of past gradients and squared gradients.

Adam Algorithm Adam adjusts learning rates for each weight individually using the following steps:

$$
\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t \\
v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \\
\hat{m}_t &= \frac{m_t}{1-\beta_1^t} \\
\hat{v}_t &= \frac{v_t}{1-\beta_2^t} \\
W_t &= W_{t-1} + \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}
\end{aligned}
$$

where g_t is the gradient at time step t, $\beta_1 = 0.9$ and $\beta_2 = 0.999$ are decay rates, and $\epsilon = 10^{-8}$ is a small constant to prevent division by zero.

Results Based on the grid search, the best configuration was: α: **0.5**; hidden: **8**; activation: **ReLU**; L_2 penalty: 0.0; learning rate schedule: **geometric**; best epoch: **33**.

Table 4 shows the outcomes for each tested combination.

Table 4: Adam results: testing L2 regularization and learning rate schedules.

alpha	hidden	activation	momentum	l2	lr_sch	est_CE	val_CE	best_epoch
0.5	8	relu	0.0	0.0	geometric	0.02859375	0.03	33
0.5	8	relu	0.0	0.0	step	0.05796875	0.04625	87
0.5	8	relu	0.0	0.0	constant	0.0896875	0.074375	53
0.5	8	relu	0.0	0.0	linear	0.14984375	0.148125	42
0.5	8	relu	0.0	0.01	step	0.3053125	0.255	3
0.5	8	relu	0.0	0.01	linear	0.311875	0.264375	3
0.5	8	relu	0.0	0.01	constant	0.3103125	0.28375	4
0.5	8	relu	0.0	0.01	geometric	0.30078125	0.294375	3
0.5	8	relu	0.0	0.1	constant	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	linear	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	geometric	0.45390625	0.445625	1
0.5	8	relu	0.0	0.1	step	0.45390625	0.445625	1

The final results on the test set were: **Test CE**: 1.95%, **Test RE**: 0.0345.

Confusion Matrix The confusion matrix (in %) shows the classification accuracy across three classes:

True / Pred	A	B	C
A	95.77	3.52	0.70
B	1.59	96.98	1.43
C	0.74	0.00	99.26

Table 5: Confusion Matrix.

Figures 6,7,8 below shows the results from final Adam testing. The model successfully classifies the test data into correct classes with good accuracy.

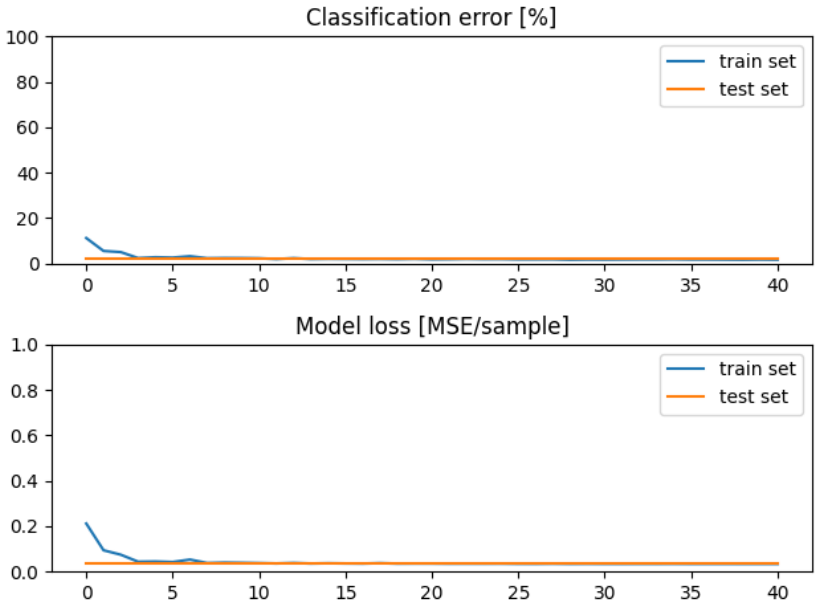

Figure 6: Error metrics during Adam training.

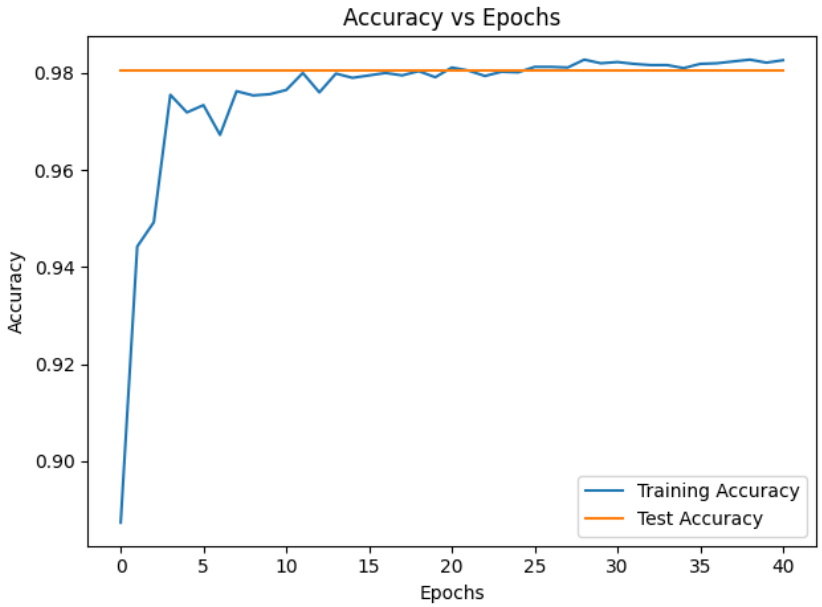

Figure 7: Accuracy per epoch with Adam optimizer.

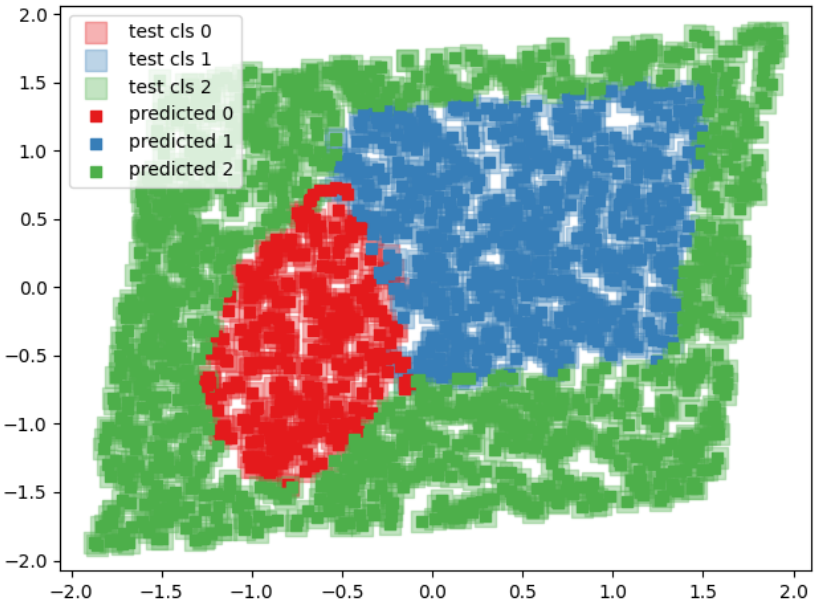

Figure 8: Adam classification on test data.